

Polygons

Paul Brandstetter
Matrikelnummer: k12310371

Abgabedatum: 7. Mai 2026

Programmieren 2

1 Quellcode

Der Code ist in 3 Files aufgeteilt. Insgesamt $59 + 193 + 227 = 499$ Zeilen Code

Klassen und Member Function Declaration sowie One-Liner definitionen sind in Poylgon.h

```
/*
 * Polygon.h
 * Declares classes Polygon, RegularPolygon, Hexagon, Square and Picture
 * also declares their methods and a handful of implementations if they are jsut one line.
 */

struct Vertex {

    double posX;
    double posY;

    struct Vertex *next;

};

class Polygon {
    /*
     * class Polygon
     * a Polygon is a linked list of Vertices that each have an x and y coordinate.
     * it is closed, that means the last vertex is connected to the first.
     * it also stores its color in hexadecimal
     */
protected:
    Vertex *points = nullptr;
    unsigned int color = 0x0;

public:
    // create polygon in denoted color (default black)
    Polygon(unsigned int color = 0);
    // copy constructor, copy assignment operator, destructor
    Polygon(Polygon &p);
    Polygon& operator=(Polygon &p);
    virtual ~Polygon();
    // create a heap-allocated copy of this polygon
    virtual Polygon* clone();
    // add point with relative coordinates (x,y) to polygon
    void add(double x, double y);
    // draws the polygon at absolute coordinates (x0,y0) scaled by factor f;
    // thus every point (x,y) is drawn at position (x0+x*f, y0+y*f)
    virtual void draw(double x0 = 0, double y0 = 0, double f = 1);
};
```

```

class RegularPolygon: public Polygon {
    /*
     * class RegularPolygon
     * a RegularPolygon is a Polygon that looks like a convex, regular polygon.
     * Additionally, it has a center, which is also drawn as a point.
     * Represented by a linked List.
     */
protected:
    double centerX = 0;
    double centerY = 0;
public:

    /*
     * Default Constructor
     */
    RegularPolygon() {
    }
    ;
    /*
     * RegularPolygon(x,y,r,n,a,c)
     * Creates a regular Polygon
     * x and y are the position of the center
     * r is the radius, so the distance from the center of a vertex
     * n is the number of vertices
     * a is the angle of the vector from center to the first vertex. Default 0
     * c is the color in hexadecimal. Default Black (=0)
     */
    RegularPolygon(double x, double y, double r, int n, double a = 0,
        unsigned int c = 0);
    /*
     * Copy Constructor
     */
    RegularPolygon(RegularPolygon &p);
    /*
     * void draw(x0,y0,f)
     * draws the polygon.
     * has to be called after beginDrawing()
     * x0 and y0 are x and y offset.
     * f is scaling factor
     */
    virtual void draw(double x0 = 0, double y0 = 0, double f = 1);

    /*
     * Destructor
     */
    virtual ~RegularPolygon();

```

```

    /*
    * RegularPolygon* clone()
    * creates a heap-allocated copy
    */
    virtual RegularPolygon* clone();
};

class Square: public RegularPolygon {
    /*
    * class Square
    * a Square is a special case of a RegularPolygon.
    * it has 4 vertices
    */
public:

    /*
    * Square(x,y,r,n,a,c)
    * Creates a Square
    * x and y are the position of the center
    * r is the radius, so the distance from the center of a vertex
    * a is the angle of the vector from center to the first vertex. Default 0
    * c is the color in hexadecimal. Default Black (=0)
    */
    Square(double x, double y, double r, double a = 0, unsigned int c = 0) :
        RegularPolygon(x, y, r, 4, a, c) {
    }
    /*
    * Square(p)
    * Copy constructor
    */
    Square(Square &p) :
        RegularPolygon(p) {
    }
    ;
    ;
};

class Hexagon: public RegularPolygon {
    /*
    * class Hexagon
    * a Hexagon is a special case of a Regularpolygon.
    * it has 6 vertices
    */
public:

```

```

/*
 * Hexagon(x,y,r,a,c)
 * Creates a Hexagon
 * x and y are the position of the center
 * r is the radius, so the distance from the center of a vertex
 * a is the angle of the vector from center to the first vertex. Default 0
 * c is the color in hexadecimal. Default Black (=0)
 */
Hexagon(double x, double y, double r, double a = 0, unsigned int c = 0) :
    RegularPolygon(x, y, r, 6, a, c) {
}
/*
 * Hexagon(p)
 * Copy Constructor
 */
Hexagon(Hexagon &p) :
    RegularPolygon(p) {};
};

class Picture {
/*
 * class Picture
 * a Picture is a linked list. Each Element contains a Polygon as Data.
 */
Polygon *data = nullptr;
Picture *next = nullptr;
public:

/*
 * Default Constructor
 */
Picture() {
}
/*
 * Copy constructor
 */
Picture(Picture &p);
/*
 * Copy assignment operator
 */
Picture& operator=(Picture &p);
/*
 * Destructor
 */
~Picture();

```

```

/*
 * void add(p)
 * Adds Polygon p to the linked List that is your Picture
 */
void add(Polygon &p);

/*
 * draw(x,y,w,h,f)
 * Creates a Canvas of width w and height h.
 * Draws all the polygons in the picture from position (x,y) with scaling factor f.
 */
void draw(double x, double y, double w, double h, double f = 1.0);
};

```

Implementierung der Member Functions sind in Polygon.cpp

```

//cpp file for "Polygon.h"
#include "Polygon.h"
#include "Drawing.h"
#include <cmath>

Polygon::Polygon(unsigned int color) {
    this->color = color;
    this->points = nullptr;
}

Polygon::Polygon(Polygon &p) {
    this->color = p.color;
    if (p.points == nullptr) {
        this->points = nullptr;
        return;
    }

    Vertex *other = p.points;
    Vertex *tmp = new Vertex;
    this->points = tmp;
    while (true) {
        tmp->posX = other->posX;
        tmp->posY = other->posY;

        if (other->next == nullptr) {
            tmp->next = nullptr;
            return;
        }

        tmp->next = new Vertex;
    }
}

```

```

        other = other->next;
        tmp = tmp->next;
    }
}

Polygon& Polygon::operator=(Polygon &p) {
    this->color = p.color;
    if (p.points == nullptr) {
        this->points = nullptr;
        return *this;
    }

    Vertex *other = p.points;
    Vertex *tmp = new Vertex;
    this->points = tmp;
    while (true) {
        tmp->posX = other->posX;
        tmp->posY = other->posY;

        if (other->next == nullptr) {
            tmp->next = nullptr;
            return *this;
        }

        tmp->next = new Vertex;

        other = other->next;
        tmp = tmp->next;
    }
    return *this;
}

Polygon::~~Polygon() {
    Vertex *curr = this->points;
    Vertex *tmp;
    while (true) {
        if (curr == nullptr) {
            return;
        }
        tmp = curr->next;
        delete curr;
        curr = tmp;
    }
}

Polygon* Polygon::clone() {

```

```

    Polygon *clone = new Polygon(*this);
    return clone;
}

void Polygon::add(double x, double y) {
    if (this->points == nullptr) {
        this->points = new Vertex;
        this->points->posX = x;
        this->points->posY = y;
        this->points->next = nullptr;
        return;
    }
    Vertex *tmp = this->points;
    while (true) {
        if (tmp->next == nullptr) {
            tmp->next = new Vertex;
            tmp->next->posX = x;
            tmp->next->posY = y;
            tmp->next->next = nullptr;
            return;
        }
        tmp = tmp->next;
    }
}

void Polygon::draw(double x0, double y0, double f) {
    if (this->points == nullptr) {
        return;
    }
    Vertex *tmp = this->points;
    double startX = 0, startY = 0, endX = 0, endY = 0;
    while (true) {
        startX = x0 + tmp->posX * f;
        startY = y0 + tmp->posY * f;
        compsys::fillEllipse(startX - 2, startY - 2, 4, 4, this->color);

        if (tmp->next == nullptr) {
            break;
        }

        tmp = tmp->next;
        endX = x0 + tmp->posX * f;
        endY = y0 + tmp->posY * f;
        compsys::drawLine(startX, startY, endX, endY, this->color);
    }
}

```



```

        compsys::drawLine(endX, endY, x0 + this->points->posX * f,
            y0 + this->points->posY * f, this->color);
    }

RegularPolygon::RegularPolygon(double x, double y, double r, int n, double a,
    unsigned int c) {
    centerX = x;
    centerY = y;
    color = c;
    points = nullptr;
    const double pi = 3.14159265358979323846;

    double offsetX, offsetY;
    for (int i = 0; i < n; i++) {
        offsetX = r * std::cos(2 * pi * i / n + a);
        offsetY = r * std::sin(2 * pi * i / n + a);
        this->add(x + offsetX, y + offsetY);
    }
}

RegularPolygon::RegularPolygon(RegularPolygon &p) :
    Polygon::Polygon(p) {
    this->centerX = p.centerX;
    this->centerY = p.centerY;
}

void RegularPolygon::draw(double x0, double y0, double f) {
    Polygon::draw(x0, y0, f);
    compsys::fillEllipse(x0 + f * this->centerX - 2, y0 + f * this->centerY - 2,
        4, 4, this->color);
}

RegularPolygon::~RegularPolygon() {
    //Nothing to be done here as the linked list is handled by Polygons destructor.
}

RegularPolygon* RegularPolygon::clone() {
    RegularPolygon *clone = new RegularPolygon(*this);
    return clone;
}

Picture::~Picture() {
    if (this == nullptr) {
        if (this->data != nullptr) {
            delete this->data;
        }
    }
}

```

```

        return;
    }
    delete this->next;
}

Picture::Picture(Picture &p) {
    //can this be recursive??
    this->data = p.data->clone();
    if (p.next == nullptr) {
        this->next = nullptr;
        return;
    }
    this->next = new Picture(p);
}

Picture& Picture::operator=(Picture &p) {
    this->data = p.data->clone();
    if (p.next == nullptr) {
        this->next = nullptr;
        return *this;
    }
    this->next = new Picture(p);
    return *this;
}

void Picture::add(Polygon &p) {
    if (this->next == nullptr && this->data == nullptr) {
        this->data = p.clone();
        return;
    }
    Picture *tmp = this;
    while (true) {
        if (tmp->next == nullptr) {
            tmp->next = new Picture;
            tmp = tmp->next;
            tmp->data = p.clone();
            return;
        }
        tmp = tmp->next;
    }
}

void Picture::draw(double x, double y, double w, double h, double f) {
    compsys::beginDrawing(w, h, "Picture");
    if (this->next == nullptr && this->data == nullptr) {
        compsys::endDrawing();
    }
}

```

```

        return;
    }
    Picture *tmp = this;
    while (true) {
        tmp->data->draw(x, y, f);
        if (tmp->next == nullptr) {
            compsys::endDrawing();
            return;
        }
        tmp = tmp->next;
    }
}

```

int main sowie alle Tests sind in PolygonMain.cpp

```

#include "Drawing.h"
#include "Polygon.h"
#include <iostream>

#include <cmath>

int main() {

    //Some Tests that I had in there when
    //trying to get copy onstructor, copy assignment operator etc to work.
    Polygon p = Polygon(0x000000);
    p.add(10, 10);
    p.add(25, 0);
    Polygon q;
    //Shows that my copy assignment operator works
    q = p;
    Polygon *r = p.clone();
    r->add(600, 500);
    q.add(30, 30);
    compsys::beginDrawing(700, 700, "Polygons");
    p.draw(300, 300, 8);
    q.draw(200, 300, 8);
    RegularPolygon x(100, 100, 50, 5);
    x.draw(100, 100, 3);
    compsys::endDrawing();

    //Septagons
    Picture test;
    for (int i = 0; i < 32; i++) {
        RegularPolygon a(0, 5 * i, 200, 7, i, 8 * i * 0x010000);
        test.add(a);
    }
}

```

```

test.draw(300, 200, 600, 600);

//Hexagons and Squares
Picture test1;
for (int i = 0; i < 10; i++) {
    Square a(400, 400, 50 * i, i, 0x000000);
    Hexagon b(400, 400, 50 * i, i, 0xFFBBBB);
    test1.add(a);
    test1.add(b);
}
test1.draw(0, 0, 800, 800);

//Snake approximates a lissajous curve. First it's drawn in the center, then in the corner at 4 t
Polygon snake;
for (int i = 0; i < 120; i++) {
    const double pi = 3.14159265358979323846;
    snake.add(100 * std::sin(i / pi), 100 * std::cos(3 * i / pi));
}
Picture test2;
test2.add(snake);
test2.draw(400, 400, 800, 800, 1);
Picture test3;
test3 = test2;
test3.draw(200, 200, 800, 800, 4);

return 0;
}

```

2 Tests

Es wurde nicht mit Inputs getestet. Es gab keinen Textoutput an die Console. Allerdings wurden 5 Canvas ausgegeben ausgegeben Das Programm terminiert mit returnvalue 0.

Bild 1: Hier wurden einige Polygone getestet und direkt gezeichnet.

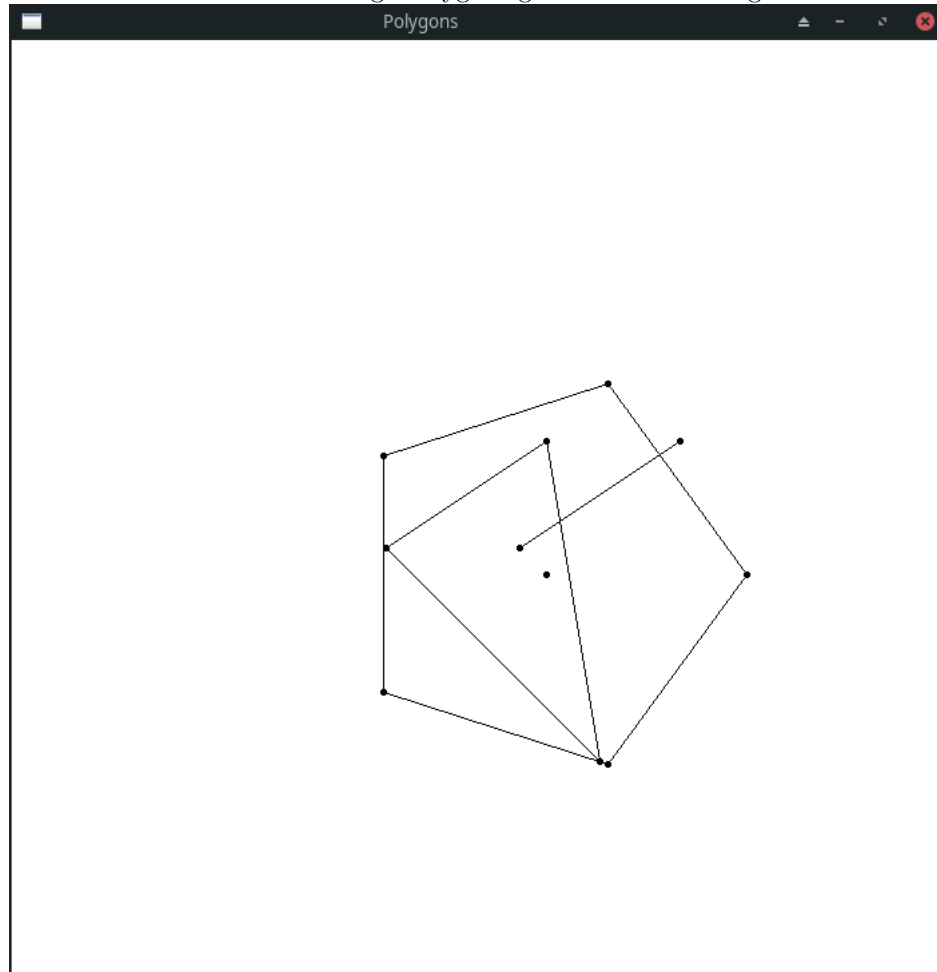


Bild 2: Hier wurden 32 gleichmäßige Siebenecke in ein Picture geladen und das bild wurde gezeichnet. Sie sind alle gleich groß, aber gestaffelt und haben andere rotation und farbe. Man sieht großartig, dass bei einem gleichmäßigen Vieleck der Mittelpunkt extra gezeichnet wird.

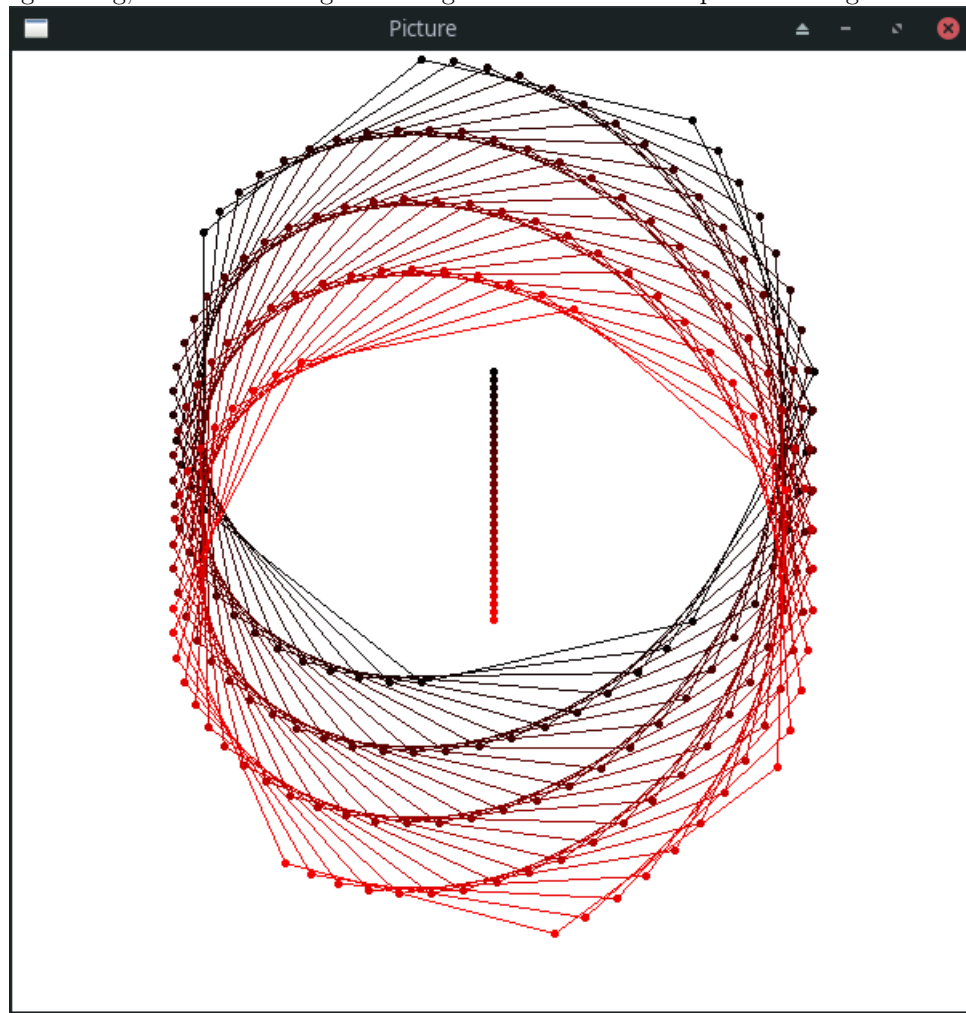


Bild 3: Hier wurden einige Quadrate(in schwarz) und Sechsecke (in rosa) in ein Picture geladen.
Sie sind alle konzentrisch, also ist der Mittelpunkt rosa, wie das letzte Polygon.

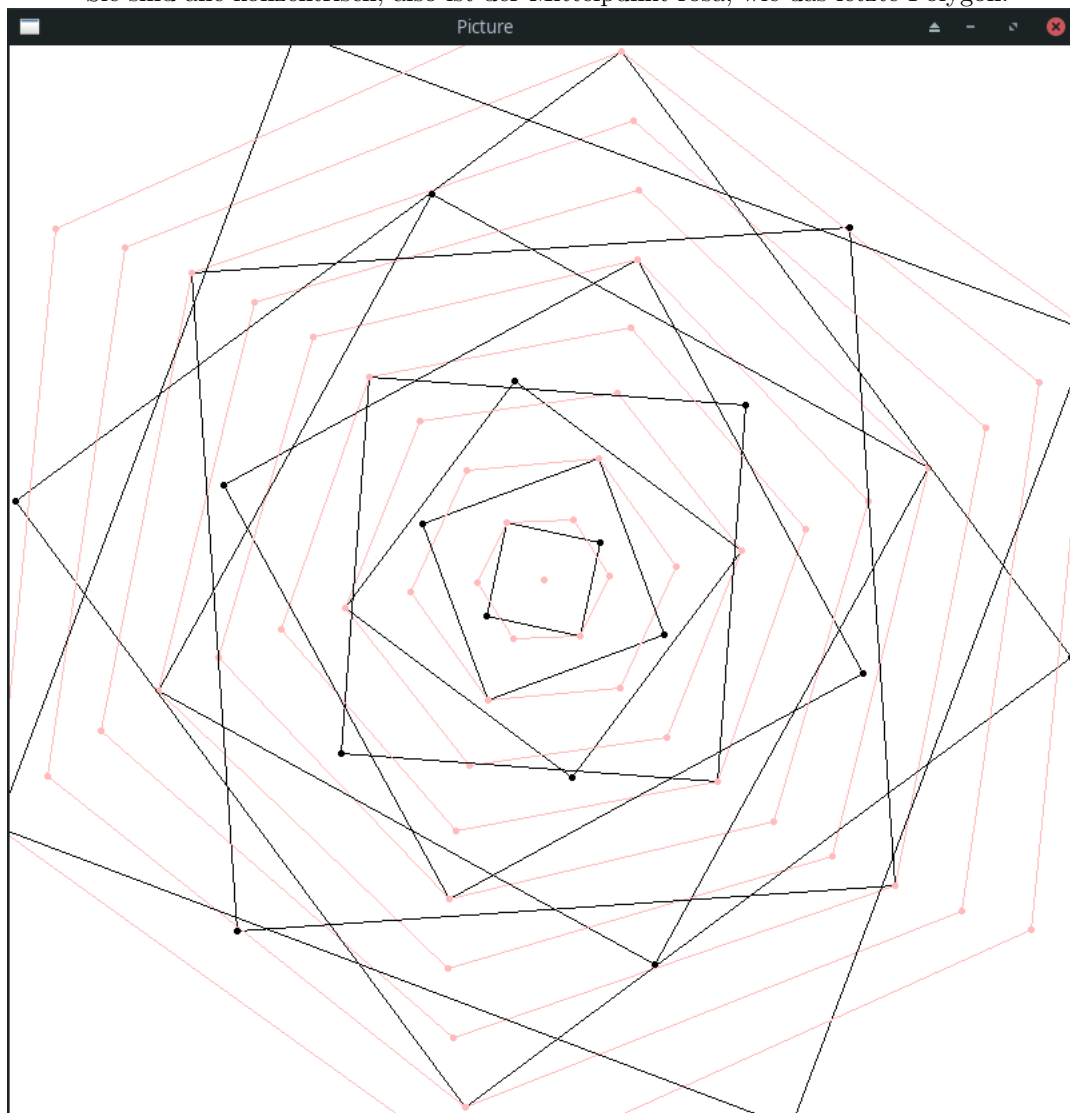


Bild 4: Hier wurde eine Lissajous-Kurve von einem Polygonzug approximiert. Diese wurde in ein Picture geladen und gezeichnet.

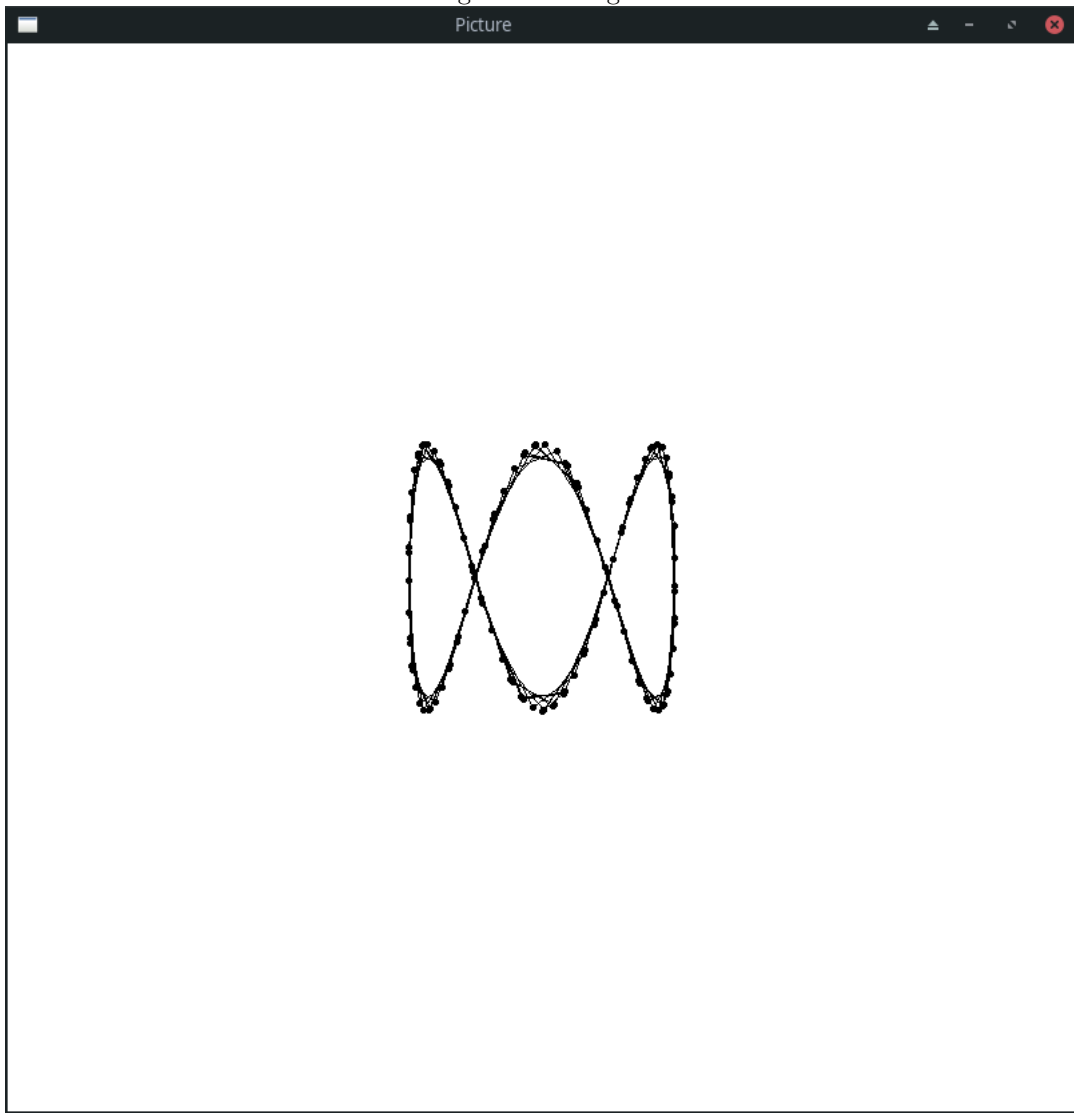


Bild 5: Hier wurde das Picture von eben kopiert, aber an einem anderen Ort mit anderem Skalierungsfaktor gezeichnet.

